

Zomato LLD



⊛ Design a Food Delivery App → (Tomato)

⊛ Gathering Requirements

step 1 :→

⊙ Functional Requirement

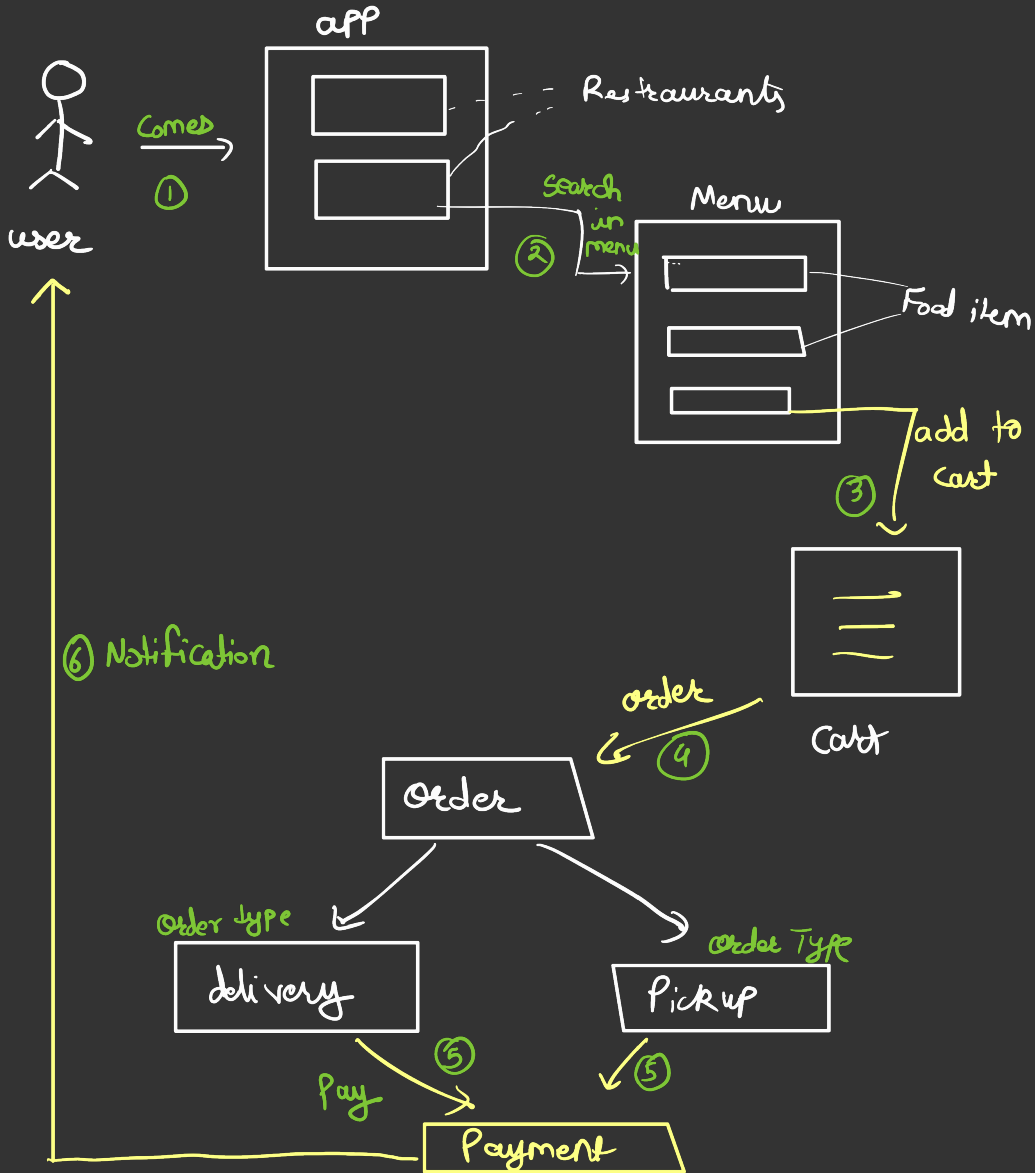
- user can search for Restaurants based on Location
- user can add items to cart
- user can checkout by Making Payment
- user should be notified once order is placed successfully.

⊙ Non functional Requirement

- ↳ Each part of design should be scalable and modifiable.

Step 2 :→ Discuss Happy flow to narrow down requirements further

Happy Flow :→



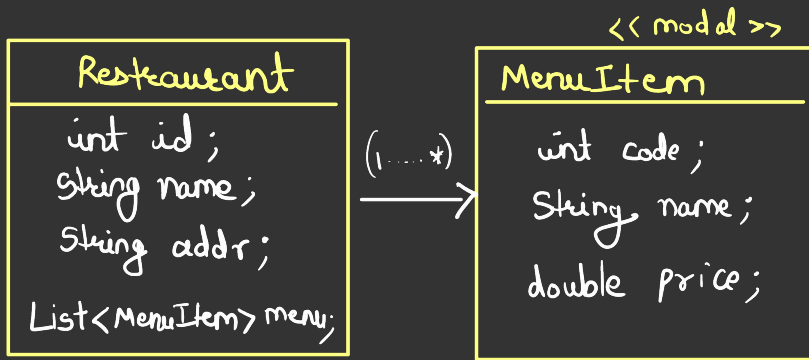
Ask, if it is 3rd party or needs to be implemented by us.

Ask, Food delivery app should be user centric or delivery agent centric. [The above diagram is user centric]

In this case, we will focus User Centric View.

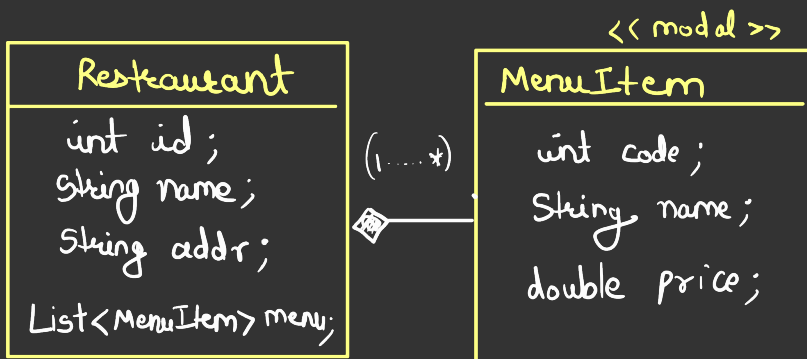
(#) Now after discussing happy flow, interviewer & we are on same page. Now start with UML.

(*) UML ; $\rightarrow$ Bottom-up approach [small to Big]

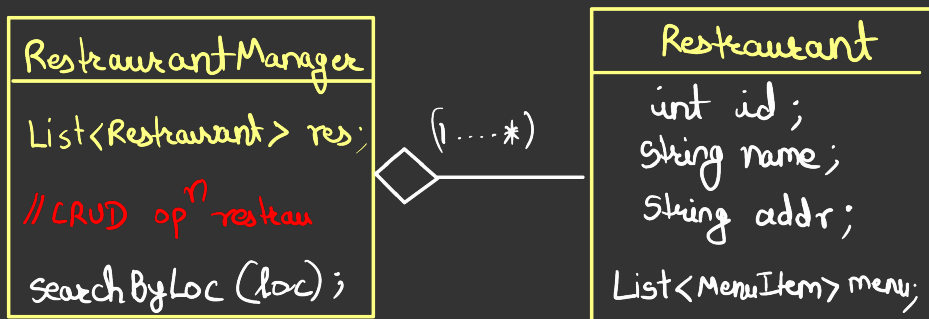


It is "has-a" relationship but which one?

$\rightarrow$ Menu Item cannot exist independently without Restaurant in this case so, it is Composition.



So, there are multiple Restaurants, so there should be a class to manage Restaurants.

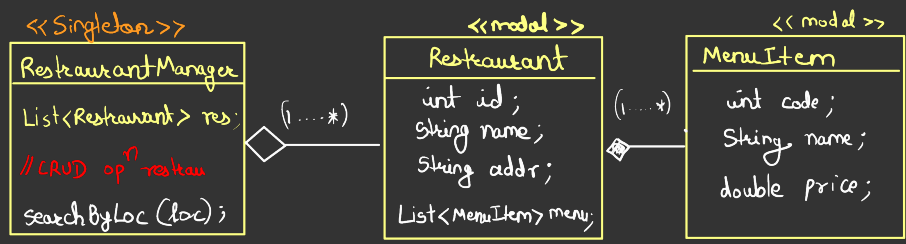


Generally, ye jo manager objects hote hai na, jo dusri class ko manage karne ka kaam karte hai, vo single point of contact hote hai for that object.

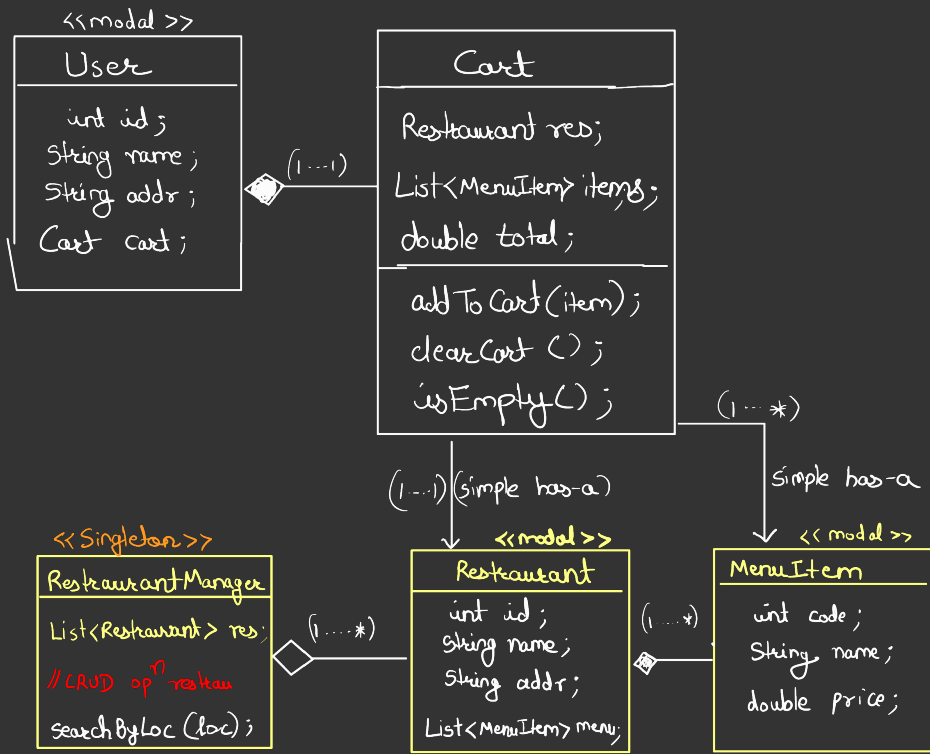
Can exists without Restaurant Manager but it comes under Restaurant manager, so it is aggregation.

Restaurant object se interact via RestaurantManager hi hoga. } Managers will always be singleton class

So far, One Component is ready.

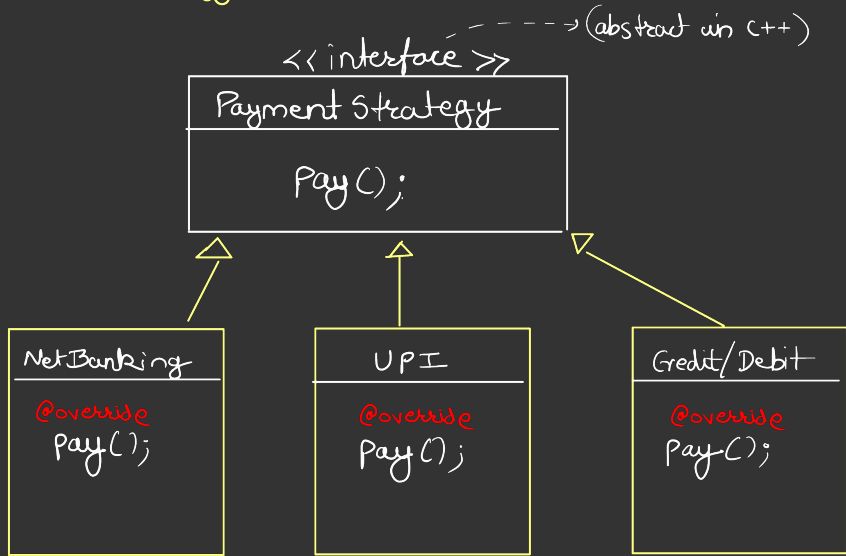


Let's create a user, who will come to restaurant manager



* Now, Let's Create Payment Flow → (this is bottom up approach)

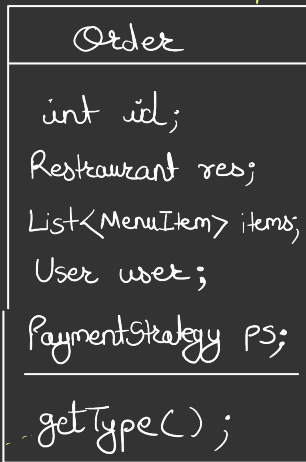
→ We will use Strategy design Pattern for Payment Flow



③ Revisit complete flow, Order is pending!!

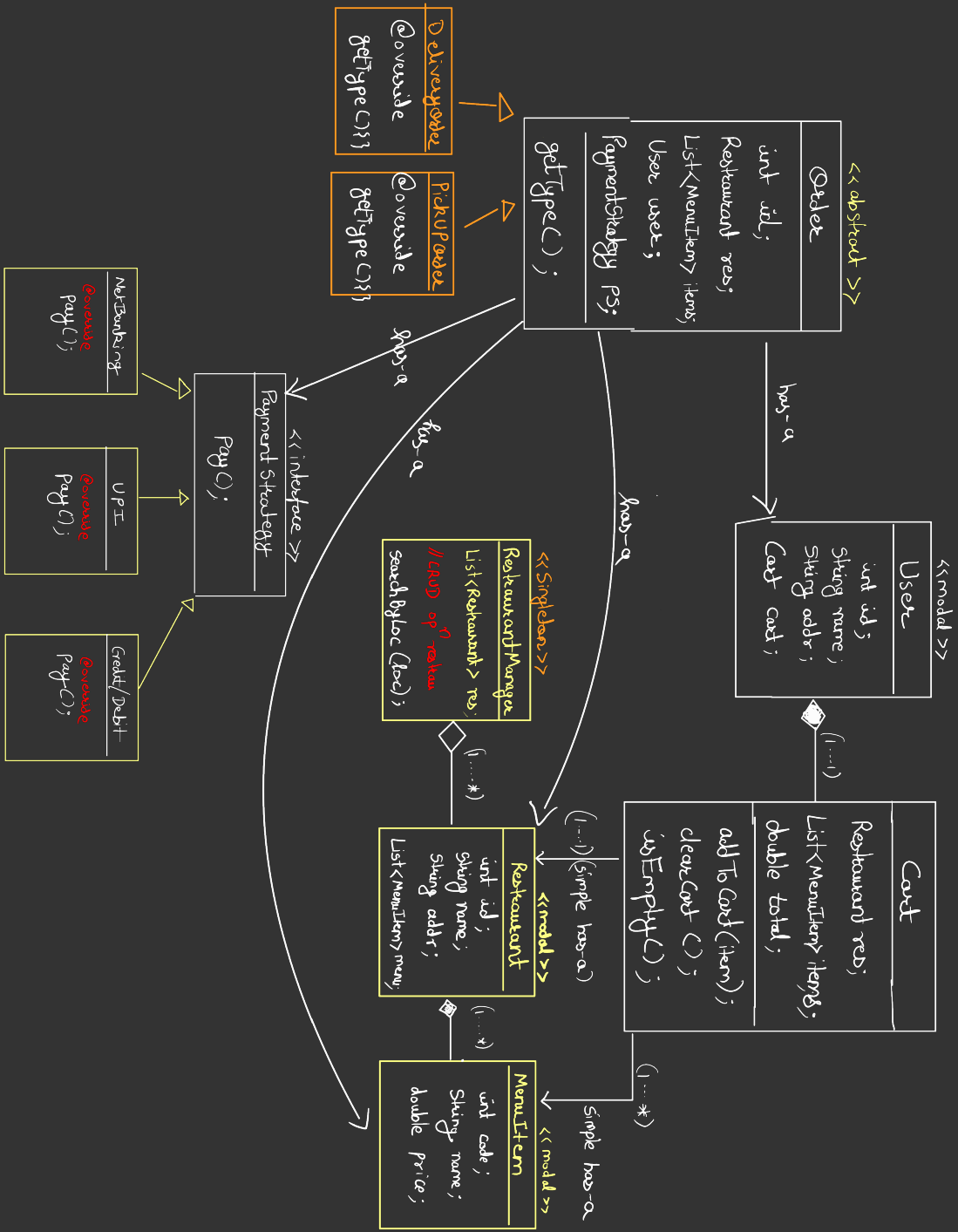


<<abstract>> , since we have abstract method.



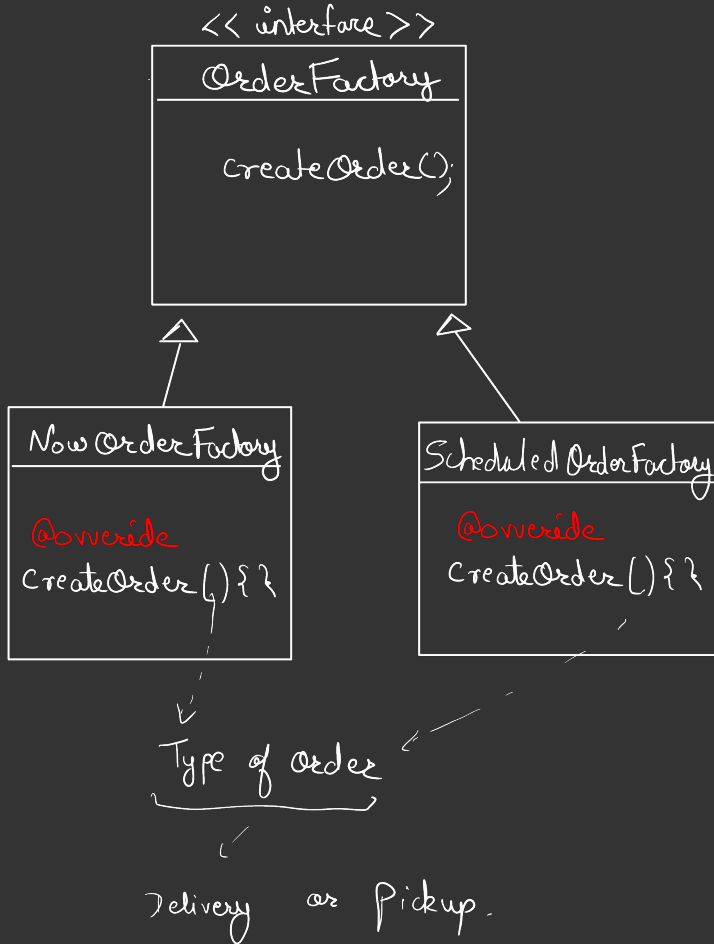
abstract
method.

② Relationships of Order

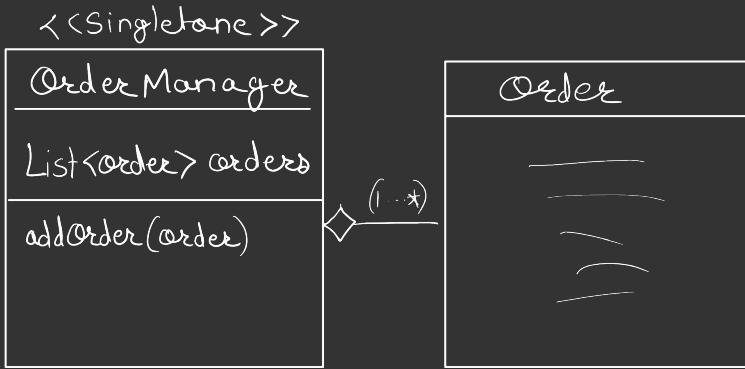


- ① Now, we know order object will be created again & again when user places order. So we should separate order object creation logic using Factory Design Pattern.
- ② We might have done same thing in case of Restaurant object, To create objects via RestaurantManager we might have used Factory design pattern to make app scalable. This can be discussed with interviewer.

→



→ Now let's create OrderManager.

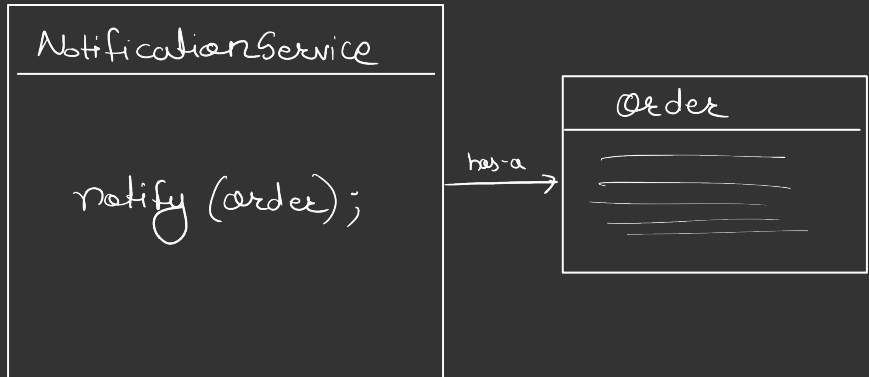


* One last thing is remaining := Notification after order

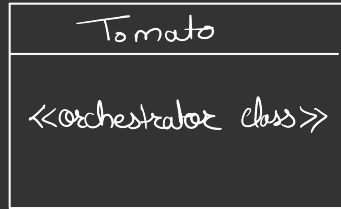
⇒ Ask interviewer, should we use 3rd Party or our own Implⁿ

Here we will use 3rd Party as Notificatⁿ system is itself LLD question

Assume 3rd Party notification system is already implemented, we just have to call it.



⑧ Finally ek class banne chahiye jo is Pure design ko hold kare. That class is called "Orchestrator class"



Is class ka kaam hai, sare classes se interact karna aur ek single Point of Contact dena "Client" ko.

This breaks SRP & least knowledge principle so it is not perfect design but more than enough in interview.

